DA test plan

Your testing

Start when you're ready to begin the first unfinished step. Overall test time: **00:00:00**

Your name

GitHub name (optional)

DA version tested

2026.10.93

Began testing

yyyy-mm-dd

Finished testing

yyyy-mm-dd

0 / 449 steps done

0 passed, 0 skipped, 0 failed, 449 to go

▶ Earlier test runs

Thank you for testing DA. This plan walks through everything DA does, step by step: first with the **menu** (what most people use), then with **commands typed in a terminal** (for things the menu doesn't have). Each step says what to do and what you should see. Mark it **passed** when it works, **skipped** when you can't try it, and **failed** when it doesn't: write down the step's number (like **D7**) and what happened, and see Reporting a problem.

On the website this page keeps track for you: fill in your name at the top (and the version, which is filled in already), then mark each step with its buttons: ✓ **passed** (green), ○ **skipped** (yellow) or ✗ **failed** (red); press it again to clear it. The page counts your progress and remembers it - in this browser, on this computer only. Each step's **Report** button opens a new problem report with the step, your name and the version already filled in, and marks the step failed. Problems already reported for a step (by you or anyone) are listed under it, with whether they're still open and their labels. **Save as PDF** keeps a dated copy of your results; when a new version of DA comes out, **Start again on a new version** clears them for the next round. On paper, use the printable checklist.

The website has one **Start timing** button. When you're ready, start it; the first unfinished step is timed. Choose that step's passed, skipped or failed result to save its time and automatically move timing to the next unfinished step. Use **Pause timing** for breaks; resume when you're ready. Each section's time is the sum of its steps, and the overall time is the sum of all section times. Timing is saved in this browser along with your results; starting again on a new version clears it.

You don't need to do it all in one sitting: each part starts from where the one before left off. The whole plan takes a few evenings. The user guide explains anything this plan mentions; the known problems list what's already known, so you don't need to report those.

Before you start

- **Use copies, never your only photos.** DA never deletes anything and asks before it moves anything, but this is a test: copy some photos into a new folder and test with those.
- **A Windows PC** with about 5 GB free, and an internet connection for a few steps (places, updates).
- **Write down your DA version** (it's shown in step A5) when you report anything.
- Words used here: an **archive** is the folder DA looks after; **intake** is where you put new files; a **sidecar** is the small `.meta.json` file DA keeps beside each photo; the **menu** is what opens when you double-click an archive's `Open DA.cmd` (or type `da`).

Make a test kit

Make a folder `C:\DA-test-kit` and put copies of these in it. The more kinds you have, the more of the plan you can try; skip a step whose files you don't have and mark it **skipped**.

Folder in the kit	What to put in it	Used in
`phone`	20-50 photos from a phone (JPG or HEIC), some taken outdoors (they have a location)	C, H, K, L
`phone`	2-3 iPhone **Live Photos** (each is a `.HEIC` or `.JPG` plus a `.MOV` with the same name)	I
`phone`	2-3 ordinary videos (`.mov` or `.mp4`, longer than 5 seconds)	I
`camera`	5-10 photos from a different camera (another phone, or a digital camera)	H, K
`scans`	5 scans of old prints (JPG/PNG/TIFF that have **no date** in them or in their name)	G, J
`docs`	3-5 PDFs (bills, letters) and a Word or Excel file	G
`doubles`	3 photos that are **exact copies** of photos in `phone` (copy them again)	F
`old`	3-5 photos taken in a different decade from the rest (say 2005 if the rest are 2020s)	M
`other`	an old camera video (`.AVI` / `.MPG`), a `.CR2` / `.CR3` / `.NEF` raw file with its JPG and one without, a screenshot	D, I

A. Install DA · Time: 00:00:00 0 / 23

#	Do this	You should see	Result
A1	Open https://da.gamenightbymike.com	The DA User Guide, with a **Download DA** button and a contents list on the left	Time: 00:00:00
A2	Click **Download DA**	`DA.exe` downloads (about 75 MB); if the browser asks, choose **Keep**	Time: 00:00:00
A3	In File Explorer, go to Downloads, click the address bar, type `powershell`, press Enter	A blue/black terminal window opens in Downloads	Time: 00:00:00
A4	Type `.\DA.exe install` and press Enter	"Installed C:\Scripts\DA\DA.exe and added it to your PATH" (or "already on your PATH")	Time: 00:00:00
A4a	If instead you see "*an application control policy has blocked this file*" or a Windows Security warning that won't go away	Go to step **A10** (install with Python) and skip A5-A9	Time: 00:00:00
A4b	If you see "*Windows protected your PC*"	Click **More info**, then **Run anyway**; then A4 works	Time: 00:00:00
A4c	At the end of A4, read the *Long paths* line	*Long paths: on*, or *Long paths are off* and *Turn long paths on now?*: answer **y**, click **Yes** in Windows' permission window; it says *Long paths: on*	Time: 00:00:00
A4d	In File Explorer, right-click `C:\Scripts\DA\DA.exe`, **Properties**, **Details**	*File version* and *Product version* show the same version as `da version` (not 0.0.0.0)	Time: 00:00:00
A5	Close the terminal, open a **new** one (Windows key, type `powershell`, Enter), type `da version`	`DA v2026.10.…` - write this version down	Time: 00:00:00
A6	Type `da --help`	A list of commands (setup, run, status, archives, guide, ...)	Time: 00:00:00
A7	Type `da guide`	The user guide opens in your browser	Time: 00:00:00
A8	Type `da guide --pdf`	The guide's PDF opens	Time: 00:00:00
A9	Type `da update --check`	*This is DA v...*, then *You have the newest DA* (or *DA v... is out*)	Time: 00:00:00
A9a	*(When a newer DA is out)* `da update`, answer **y**	*Downloading*, *its size and fingerprint match*, *Put in place: C:\Scripts\DA\DA.exe*, *Updated to DA v...*; a new terminal's `da version` shows it	Time: 00:00:00

#	Do this	You should see	Result
A9b	*(When a newer DA is out)* Open the menu (`da`)	A yellow line under the status: *DA v... is out (you have v...): da update*	Time: 00:00:00
A10	*(Python install - only if Windows blocks DA.exe, or to test it)* Follow the guide's Install DA with Python: install Python from python.org	`py --version` in a new terminal shows `Python 3.…`	Time: 00:00:00
A11	Type the guide's first line: `py -m venv C:\Scripts\DA\python`	No error; the folder `C:\Scripts\DA\python` exists	Time: 00:00:00
A12	Type the second line (`…python.exe -m pip install -r https://da.gamenightbymike.com/python/requirements.txt`)	It downloads and ends with "Successfully installed … devmukmuk-da-…"	Time: 00:00:00
A13	Type the third line (`…python.exe -m mindit install`)	"Installed C:\Scripts\DA\da.cmd (DA run by Python …)"; if a DA.exe was there: "Set aside DA.exe (now DA.exe.off)"	Time: 00:00:00
A14	New terminal: `da version`	The version, with **no** Windows Security pop-up	Time: 00:00:00
A15	`da update`	*You have the newest DA*, or (when one is out) *Downloading and installing it* … then *Updated to DA v...*	Time: 00:00:00
A16	Outside any archive, type `da status`	"Not inside an archive" and how to open one (no crash)	Time: 00:00:00
A17	Type `da` on its own (outside any archive)	"You're not inside an archive" and an offer to create one (answer **n** for now)	Time: 00:00:00

B. Create an archive · Time: 00:00:00 0 / 23

#	Do this	You should see	Result
B1	In a terminal: `da setup C:\DA-test\jane-archive-202x --years 2020-2029`	"Archive ready" and the folder's path	Time: 00:00:00
B2	Open `C:\DA-test\jane-archive-202x` in File Explorer	`da-archive.yaml`, `Open DA.cmd`, and folders `intake` (with `photos`, `docs`, `scans`), `photos`, `documents`, `output`, `logs`	Time: 00:00:00
B3	Open `da-archive.yaml` in Notepad (right-click, Open with, Notepad), then close it	`name: "jane-archive-202x"`, `years: "2020-2029"`, the intake folders, `contact_sheets: layout: 3x4`	Time: 00:00:00
B4	Double-click `Open DA.cmd`	A window opens: a box with the archive's name, folder and years, then the **Status** (intake: 0 files), then the menu keys	Time: 00:00:00
B5	Look at the menu	Keys **1-7**, **L**, **S**, **M**, **R**, **D**, **J**, **U**, **H**, **Q**	Time: 00:00:00
B6	Press **H**, Enter	A short "How DA works" help, and a list of commands	Time: 00:00:00
B7	Press **U**, Enter	The user guide opens in the browser	Time: 00:00:00
B8	Press **x** (not a menu key), Enter	"Pick one of the numbers or letters shown." and the menu again	Time: 00:00:00
B9	Press **Q**, Enter	The menu closes; the window stays open with a line saying you can type `da` for the menu	Time: 00:00:00
B10	Type `da` in that window	The menu again, starting with the Status	Time: 00:00:00
B11	Press **7** (New archive), Enter	"Archives on this PC already:" with `jane-archive-202x … <- open now`, then a question for a new folder	Time: 00:00:00
B12	Type `C:\DA-test\sam-archive-200x`, Enter; years `2000-2009`, Enter	"Archive ready", and the menu now shows **sam-archive-200x** with its Status	Time: 00:00:00
B13	Press **6** (Switch archive), Enter	Both archives listed with their years; type the number of **jane-archive-202x**	Time: 00:00:00
B14		The menu shows jane-archive-202x again, with its Status	Time: 00:00:00

#	Do this	You should see	Result
B15	Press Q. In the same window type `da archives`	A table of this PC's archives (name, years, folder)	Time: 00:00:00
B16	`da archives --open 2`	The second archive's menu opens (press Q to leave)	Time: 00:00:00
B17	`da setup C:\DA-test\sam-archive-200x --years 2000-2009` again	It says the archive is already there and keeps its settings (nothing moved)	Time: 00:00:00
B18	Make a folder `C:\DA-test\filled`, copy 3 kit photos into it, then `da setup C:\DA-test\filled`	It lists what's in the folder and asks before moving those into `intake\`; answer **n**	Time: 00:00:00
B19	Run it again and answer **y**	The 3 photos are now in `C:\DA-test\filled\intake\`, and the archive's files appear	Time: 00:00:00
B20	`da setup C:\DA-test\bad --years 1999-1990`	A clear message that years look like 1990-1999, 1999 or 199x (nothing made)	Time: 00:00:00
B20a	Make `C:\DA-test\one-year`, `cd` into it, `da setup --here --yes --years 1999`	An archive there; its `da-archive.yaml` says `years: "1999-1999"`	Time: 00:00:00
B20b	Make `C:\DA-test\decade`, `cd` into it, type `da`; then **h** and `200x`	*You're not inside an archive*, your archives and **H** *make this folder an archive*; then it's an archive with years 2000-2009 and its menu opens	Time: 00:00:00
B20c	`cd C:\` and `da setup --here --yes`	Refused: *the top of a drive* (nothing made)	Time: 00:00:00

C. The first run: process intake (menu 1) · Time: 00:00:00 0 / 26

Copy the whole test kit (except `old`) into `jane-archive-202x\intake\photos\kit` (so `kit\phone`, `kit\camera`, … are inside), copy `scans` into `intake\scans`, and `docs` into `intake\docs`.

#	Do this	You should see	Result
C1	Open jane-archive-202x (its `Open DA.cmd`)	The Status shows the intake files, folder by folder, with counts	Time: 00:00:00
C2	Press **1** (Process intake)	"Counting what's in intake …", then a list of intake's folders with file counts	Time: 00:00:00
C3	Type the number of `photos` to open it	Its subfolders (`kit`) are listed; you can open further, go up, or filter by name	Time: 00:00:00
C4	Type part of a name (for example `kit`) to filter	Only matching folders are listed	Time: 00:00:00
C5	Go back up to the top of intake and choose to process **all of intake**	The **steps after filing** list: 8-12, with **9 Tags** ticked	Time: 00:00:00
C5a	Type **10**, then **10** again	Places switches on, then off	Time: 00:00:00
C5b	Type **8** and press **Enter**	*(Kept as your usual steps.)*, then "Processing … DA asks before moving anything"; `da-archive.yaml` now has `run:` `after_filing: [owners, tags]`	Time: 00:00:00
C6	Watch step **1/12 Indexing**	A count of files indexed, no error	Time: 00:00:00
C7	Step **2/12 Hashing**	A progress count, then a summary	Time: 00:00:00
C8	Step **3/12** (duplicates): it asks whether to set exact copies aside; answer **y**	The 3 `doubles` are listed and moved to `_duplicates-holding` (see F)	Time: 00:00:00
C9	Step **4/12 Preview**	"planning names and folders (nothing moves)" and a plan: how many files to file, how many need review	Time: 00:00:00
C10	Step **5/12 Plan/check**	Numbers of files to file, raw files and Live Photo videos going with them, and problems to review	Time: 00:00:00
C11	It asks whether to file them; answer **n**	It stops; nothing was moved; the run is recorded in `output\runs\`	Time: 00:00:00
C12	Press **1** again and process all of intake again; answer **y** to each question	Steps 1-7 run; "Filed N; unchanged N (M re-hashed); problems: 0" and "Done. Run record: …"	Time: 00:00:00
C13		Right after, "Where things stand now:" and the Status (fewer files waiting)	Time: 00:00:00

#	Do this	You should see	Result
C14	In File Explorer open `photos\organized`	Year folders (`2021` , ...) with month folders (`2021-06` , ...)	Time: 00:00:00
C15	Open a month folder	Files named `photo_YYYYMMDD_HHMMSS_xxxx.jpg` (or `.heic`) in date order, each with a `.meta.json` beside it, and a `_contact-sheet.pdf`	Time: 00:00:00
C16	Check one photo's name against when it was taken (its date in the Photos app)	The date and time in the name match	Time: 00:00:00
C17	Open one `.meta.json` in Notepad	A text file with `"schema": {"name": "mindit.sidecar", "version": 2}` , a `file` block with its sha256, and a `camera` block	Time: 00:00:00
C18	Look in `intake\photos\kit`	Only files that need you are left (no date, other decade, ...)	Time: 00:00:00
C18a	At the end of the run	*8/12 Camera owners:* (it asks about cameras with no owner, or says all are known), *9/12 Tags: N photos tagged*, *10/12 Places: skipped*, *11/12 Location folders: skipped*	Time: 00:00:00
C18b	In a terminal: `da run --skip tags --with places` on a new folder	*9/12 Tags: skipped*, *10/12 Places:* looks up the GPS photos just filed	Time: 00:00:00
C18c	`da run --yes --with groups` on a folder with GPS photos	*11/12 Location folders:* lists the months to group (`da location group ...`), asks nothing	Time: 00:00:00
C18d	`da run --skip photos`	A clear message: *isn't a step DA can switch* with the list	Time: 00:00:00
C19	Back in the menu, press **2** (Status)	Organized photos/documents counts and the last run's time and "filed N"	Time: 00:00:00
C20	Process intake a third time	"Nothing to file" (or only the review ones are planned) - nothing is filed twice	Time: 00:00:00

D. What needs review (menu 3) · Time: 00:00:00 0 / 10

#	Do this	You should see	Result
D1	Press **3**	Groups of what the last run left, each with a reason and up to 5 examples	Time: 00:00:00
D2		**No date (in the file or its name)**: your scans of prints and any undated files	Time: 00:00:00
D3		If you included a photo from another decade in `kit` : **Taken outside this archive's years**	Time: 00:00:00
D4		"Full list: ...\review.csv"	Time: 00:00:00
D5	Open that `review.csv` (double-click; it opens in Excel or Notepad)	One row per file: reason, file, detail	Time: 00:00:00
D6	The screenshot from `other`	Filed by a date in its name, or listed under No date - not a crash	Time: 00:00:00
D7	The raw file (`.CR2` / `.NEF`)	Moved and renamed together with its JPG (same new name, raw extension), not left alone	Time: 00:00:00
D8	A raw file whose JPG isn't there (e.g. a Canon `.CR2`)	Filed as a photo by its own date: `photo_<date>_<time>_xxxx.cr2` , its sidecar has the camera; the contact sheet shows its picture	Time: 00:00:00
D8a	A Canon `.CR3` alone	The same as D8 (`…_xxxx.cr3`); the tile shows its preview the right way up	Time: 00:00:00
D8b	A `.CR3` with its `.JPG` (*RAW* + *JPEG*)	Both filed together, the same new name; one tile on the sheet	Time: 00:00:00

E. Status and the index (menu 2, 5) · Time: 00:00:00 0 / 9

#	Do this	You should see	Result
E1	Press **2**	The same numbers as the box at the top, plus intake folder by folder	Time: 00:00:00
E2	In File Explorer, move one filed photo (and its `.meta.json`) to your Desktop		Time: 00:00:00
E3	Press **5** (Refresh index)	It indexes, then the counts drop by one	Time: 00:00:00
E4	Move the photo back where it was, press **5**	Counts back up	Time: 00:00:00
E5	Terminal (in the archive): `da db stats`	Counts by year and kind	Time: 00:00:00
E6	`da db query "SELECT file_name, captured_at FROM files WHERE area = 'photos' LIMIT 5"`	Five photos with their dates	Time: 00:00:00
E7	`da db query "DELETE FROM files"`	Refused: only SELECT is allowed (nothing deleted)	Time: 00:00:00
E8	`da status`	The same Status as menu 2	Time: 00:00:00
E9	`da intake status`	Intake folder by folder: files, GB, photos, raw, documents, videos, hashed, already in organized	Time: 00:00:00

F. Duplicates and undo (menu 4) · Time: 00:00:00 0 / 11

#	Do this	You should see	Result
F1	Open `_duplicates-holding` in the archive	Today's date folder with the 3 `doubles` (exact copies), and a `moved.csv` record	Time: 00:00:00
F2	Press **4** (Undo last set-aside)	"Put 3 files back where they were (from ...)?"; answer **y**	Time: 00:00:00
F3		"Restored 3"; the doubles are back in `intake\photos\kit\doubles`	Time: 00:00:00
F4	Process intake again (menu 1, answer y)	The doubles are set aside again (they're still exact copies of filed photos)	Time: 00:00:00
F5	Press **4** and answer **n**	Nothing changes	Time: 00:00:00
F6	Delete the folder in `_duplicates-holding` yourself (they're copies)	Fine: DA doesn't need them	Time: 00:00:00
F6a	Put a duplicate back in intake and process it again (it's set aside again), then `da holding`	*N files in holding: N have their original*; `holding-check.csv` in `output\runs\<date-time>-holding\` names each original	Time: 00:00:00
F6b	Put a file of your own in `_duplicates-holding\<date>\`, then `da holding`	*1 stay ... no original*, with its name	Time: 00:00:00
F6c	`da holding --empty`, answer **y**	The copies with an original are in the Recycle Bin (with their `.meta.json`); your own file is still there; `holding-emptied.csv` lists them	Time: 00:00:00
F6d	Restore one from the Recycle Bin	It comes back where it was	Time: 00:00:00
F7	*(Optional, long paths off: skip it if A4c turned them on)* Put a copy of a kit photo in intake inside folders nested deep enough that its path is about 240 characters, and process intake	The step carries on; *Not moved ... path too long for Windows* for that copy, which stays in intake	Time: 00:00:00

G. Documents and scans · Time: 00:00:00 0 / 5

#	Do this	You should see	Result
G1	Open `documents\organized`	Year/month folders with `doc_YYMMDD_HHMMSS_xxxx.pdf` (and your Word/Excel file)	Time: 00:00:00
G2	Compare a PDF's folder with the date it was created (File, Properties in a PDF reader)	It's filed by the date inside the PDF	Time: 00:00:00
G3	The scans you put in `intake\scans`	Treated as **documents**, not photos; scans with no date wait under No date	Time: 00:00:00
G4	A PDF with flaws (an old scanner's PDF, if you have one)	Processed without a stream of red/yellow warnings on the screen	Time: 00:00:00
G5	Put a `.jpg` scan of a letter in `intake\docs` and process	It's filed under `documents`, not `photos`	Time: 00:00:00

G2. Document templates (da template, da run --template) · Time: 00:00:00

Needs 2-3 PDF bills from one company (the same kind every month) and a few other PDFs.

#	Do this	You should see	Result
GT1	`da template new <company> --from <bill 1> --from <bill 2>`	*Wrote ...\templates\.yaml: looks for '<the company's name>', reads date, account, ...*, then each sample *matched* with the values read	☐
GT2	Compare the values shown with the bills themselves	The bill date, account number and amount due are the ones printed (not a previous balance)	☐
GT3	`da template edit <company>`: add your account number under `account:` as `expect: ["<it>"]`, save, close Notepad	*reads: date, account, ...* (no mistake)	☐
GT4	`da template test <company> <a folder with the bills and other PDFs>`	Every bill *matched*; every other PDF *failed* with a reason (*no '' in it ...*); nothing moved	☐
GT5	Put the bills and the other PDFs in `intake\docs`; `da run --kind document --template <company> intake\docs -y`	A line per bill with its values; a *! read by template and failed: ...* line per other PDF; *N of M documents matched*	☐
GT6	Open `documents\organized`	Each bill in its bill date's month, named like `<company>_2025-09-15_4455_a709.pdf`; the other PDFs still in `intake\docs`	☐
GT7	Open one bill's `.meta.json`	A `template` block with every value read, and tags `template:<company>`	☐
GT8	`output\runs\<the run>\templates.csv`	One row per PDF: matched or failed, why, the values	☐
GT9	`da run --template nothing`	*No template 'nothing' in ...\templates (there:)*; nothing done	☐
GT10	Bills filed before the template (in `documents\organized\<year>` under their old names): `da run --template <company> documents\organized\<year>`	Step 5 *N filed already to rename where they are*, years by the bill dates; after it each bill is renamed in its month folder; other documents there untouched	☐
GT11	Add `folder: 'Bills\{title}\{YYYY}'` under `file:` (`da template edit`), run GT10 again	The bills move to `documents\organized\Bills\<title>\<year>\`; month folders they left empty are gone, the others' contact sheets remade	☐

H. Contact sheets · Time: 00:00:00 0 / 24

#	Do this	You should see	Result
H1	Double-click a month's `_contact-sheet.pdf`	Page 1 is a **summary**: folder name, photo/video counts, the days, GPS count, places, sidecar versions, and a **Cameras** table with page numbers	Time: 00:00:00
H2	Page 2 onwards	A section per camera: each page headed `<month> - <camera> / <owner>` (or just the camera)	Time: 00:00:00
H3		12 pictures a page (3 across, 4 down), each with its name and date underneath	Time: 00:00:00
H4	A photo taken outdoors with a phone	**GPS** in green down its left edge	Time: 00:00:00
H5	Every picture	A small circled **2** at its top right (the sidecar version)	Time: 00:00:00
H6	A video	A dark **VIDEO** tile with its length	Time: 00:00:00
H7	A Live Photo	Shown once (its photo), not twice	Time: 00:00:00
H8	Terminal: `da contact-sheets photos\organized --layout 4x5`	"Made or updated N"; that sheet now has 20 pictures a page	Time: 00:00:00
H9	`da contact-sheets photos\organized` (no layout)	Sheets go back to 3x4	Time: 00:00:00
H10	`da contact-sheets photos\organized --layout 9x9`	A clear message: a layout looks like 3x4 or 4x5	Time: 00:00:00
H11	Run H9 again	"already up to date N" (nothing made again)	Time: 00:00:00
H12	Copy one more photo into a month folder (with no sidecar) and run `da validate <that month folder>`	Its contact sheet is made again and includes the new photo	Time: 00:00:00
H13	`da contact-sheets photos\organized --force`	Every sheet made again	Time: 00:00:00
H14	In `da-archive.yaml` set `contact_sheets:` `layout: 4x5`, save, run H9	Sheets now 20 a page; set it back to `3x4` afterwards	Time: 00:00:00

#	Do this	You should see	Result
H15	`da contact-sheets documents\organized --kind document`	*N year folders with documents*; `documents\organized\<year>\_contact-sheet.pdf` in each	Time: 00:00:00
H16	Open one	Page 1: the archive's name, documents, pages, read by da ocr, secrets, dates, months and the legend; then 2 x 2 first pages	Time: 00:00:00
H17	A document page	Pages in a square (top left), schema in a circle (top right); under it File (bold), Folder and Dates in one size; a red border on a document holding a secret	Time: 00:00:00
H18	A year not read by `da ocr` yet	Blue borders, and *N documents aren't read by da ocr yet*	Time: 00:00:00
H19	`da ocr` on that year, then H15 again	The blue borders are gone; dates shown; red borders on documents with a secret	Time: 00:00:00
H20	H15 again with nothing changed	*already up to date 1*	Time: 00:00:00
H21	`da contact-sheets documents --kind letters`	Refused: *--kind is photo or document*	Time: 00:00:00
H22	Open a photo sheet, click a picture	The photo opens	Time: 00:00:00
H23	Open a document sheet, click a first page and a File line	The document opens	Time: 00:00:00
H24	Copy the archive to another drive (or use the BACKUP4 copy), open a sheet there, click a picture	The copy's file opens (the links have no drive letter)	Time: 00:00:00

I. Videos, Live Photos and HEIC · Time: 00:00:00 0 / 14

#	Do this	You should see	Result
I1	Your ordinary videos	Filed in their month as `video_YYYYMMDD_HHMMSS_xxxx.mov` (or `.mp4`) beside the photos	Time: 00:00:00
I2	Open a video's `.meta.json`	A `video` block with `duration_seconds` and `recorded_at`; `classification` says `video`	Time: 00:00:00
I3	A Live Photo (HEIC + MOV with the same name)	Both filed together: `photo_…_xxxx.heic` and `photo_…_xxxx.mov` (same name)	Time: 00:00:00
I4	Copy a Live Photo's `.MOV` alone (without its photo) into intake, process	It stays in intake (it's the short video of a Live Photo)	Time: 00:00:00
I5	Terminal: `da pair-movies`	It finds that video's filed photo and lists where it would go; nothing moves	Time: 00:00:00
I6	`da pair-movies --apply`	"Moved 1" and an Undo line; the video is now beside its photo with the same name	Time: 00:00:00
I7	`da intake undo "<the file it names>"`	The video goes back to intake	Time: 00:00:00
I7a	A photo with a voice note (`SAM_0010.JPG` + `SAM_0010.WAV`) in intake, process	Both filed: `photo_…_xxxx.jpg` and `photo_…_xxxx.wav`	Time: 00:00:00
I7b	A `.WAV` alone whose photo was filed earlier; `da pair-movies`	It's listed with its photo; `--apply` puts it beside it	Time: 00:00:00
I7c	A folder where a few photos have a date years outside the archive's (camera clock reset), process	Step 5 warns … *a camera clock that was reset?* and names the folder; those photos stay in intake	Time: 00:00:00
I7d	An old Word `.doc` (e.g. `Journal January 4th 2009.doc`) in `intake\photos\` , process	Filed in `documents\organized\2009\…` ; one with no date in its name gets the date it was created	Time: 00:00:00
I8	An old camera `.AVI` / `.MPG`	Filed by the date inside it, or listed under No date (then see J)	Time: 00:00:00
I9	`da heic-to-jpg photos\organized\<year>\<month>`	JPG copies of the HEIC photos made in a folder beside it (the HEICs stay)	Time: 00:00:00
I10	`da heic-to-jpg <a folder> --out C:\DA-test\jpgs`	The copies go to `C:\DA-test\jpgs`	Time: 00:00:00

J. Fix dates (menu D) · Time: 00:00:00 0 / 18

#	Do this	You should see	Result
J1	Press **D**	Each intake folder with undated photos, one at a time, with a count and an example name	Time: 00:00:00
J2	For the scans folder type `1998-07`, Enter	(B1's archive takes 2020-2029, so these will wait as "outside this archive's years" - that's expected)	Time: 00:00:00
J3	For another folder type `July`, Enter	"'July' isn't a date DA reads. Use 1998, 1998-07, ..." and it asks again	Time: 00:00:00
J4	Type `2021-06-15`, Enter	Accepted	Time: 00:00:00
J5	Press Enter on a folder	That folder is skipped	Time: 00:00:00
J6		"Dates set on N files." and "Process intake now?"; answer **y**	Time: 00:00:00
J7		The folder dated 2021-06-15 is filed as `photo_20210615_000000_xxxx` in `2021\2021-06`	Time: 00:00:00
J8	Terminal: `da date list`	Photos still without a sure date, folder by folder (or "Every photo ... has a sure date")	Time: 00:00:00
J9	`da date set "<a scan>" --date 2022`	"Only a year: DA files them under 2022-01 (January)...", "Date set to 2022 on 1 file"	Time: 00:00:00
J10	`da date set "<a scan>" --date "2022-03-04 14:30"`	Date set; after processing, named `photo_20220304_143000_...`	Time: 00:00:00
J11	`da date set "<a scan>" --date 2022-13`	A clear message: not a real date	Time: 00:00:00
J12	`da date clear "<that scan>"`	"Dates set by hand taken off on 1 file"	Time: 00:00:00
J13	Pick a **filed** photo; `da date shift "<it>" --by +1h`	"Dates shifted +1h on 1 file" and a `da validate ... --mode repair` line	Time: 00:00:00
J14	Run that `da validate ... --mode repair` line	The photo is renamed one hour later (and moved if the month changed)	Time: 00:00:00
J15	`da date shift "<it>" --by -1h` and repair again	Back to its original name	Time: 00:00:00

#	Do this	You should see	Result
J16	`da date shift <folder> --by +2d` / `-30m` / `+1mo` / `-1y`	Each accepted (try one, then shift back)	Time: 00:00:00
J17	`da date shift <folder> --by 1h` (no sign)	A clear message: use +1h, -30m, …	Time: 00:00:00
J18	Shift a folder that includes an undated scan	The undated one is listed as skipped: "no sure date to shift"	Time: 00:00:00

K. Tags (terminal) · Time: 00:00:00 0 / 54

#	Do this	You should see	Result
K1	In the archive: `da tag`	"Read camera details and make automatic tags for N photos in ...?"; answer **y**; "Tagged N photos"	Time: 00:00:00
K2	Open a filed photo's `.meta.json`	`auto_tags` with tags like `maker:apple`, `camera:apple-iphone-…`, `year:2021`, and `gps` for outdoor photos	Time: 00:00:00
K3	`da tag --cameras`	A table of the cameras found, with photo counts and years; it asks who used each (type a name like `jane`, or Enter to skip)	Time: 00:00:00
K4		"saved: Use camera	Time: 00:00:00
K5	`da tag --owner "<a camera from K3>" sam`	"saved: ... -> sam" and "Update the owner: tags now?"	Time: 00:00:00
K6	`da tag photos\organized\<year>\<month> beach`	"Your tags on N files: +beach"	Time: 00:00:00
K7	`da tag photos\organized\<year>\<month> -x beach`	"-beach"	Time: 00:00:00
K8	`da tag "<one photo>" grandma "summer trip"`	Tags `grandma` and `summer-trip` added to that photo	Time: 00:00:00
K9	`da findit tags photos\organized -r grandma -v`	`"match_count": 1` and that photo	Time: 00:00:00
K10	`da findit tags photos\organized -r gps -v`	The photos with a location	Time: 00:00:00
K11	`da findit tags photos\organized -x gps -v`	The photos without	Time: 00:00:00
K12	`da findit tags photos\organized -r owner:jane -v`	Photos from the camera you named in K3	Time: 00:00:00
K13	`da tag --refresh` (answer y)	Camera details read again from every photo; tags unchanged	Time: 00:00:00
K14	Open a contact sheet made after K3 (run `da contact-sheets <month>`)	The camera sections now show **/ jane**	Time: 00:00:00
K15	Copy 2-3 screenshots or photos of receipts into `intake\phone` and choose **1 Process intake**	They're filed under `photos\organized`	Time: 00:00:00

#	Do this	You should see	Result
K16	`da tag ocr photos\organised` (a typo)	*No such folder or file*; nothing is tagged	Time: 00:00:00
K17	`da tag ocr photos\organized\<their year>`	A preview: how many are mostly text (document), with some text, without; *Nothing changed*; the path of `ocr-results.csv`	Time: 00:00:00
K18	Open that `ocr-results.csv` in Excel	One row per picture: level, words, *would tag and move* for the screenshots, and the start of their text	Time: 00:00:00
K19	`da tag ocr photos\organized\<their year> --tag-only`	*read now: 0* (the preview's reading is reused); *Tagged N pictures*	Time: 00:00:00
K20	Open a screenshot's `.meta.json`	An `ocr` block with its text, and `text`, `document` in `auto_tags`	Time: 00:00:00
K21	`da tag ocr photos\organized\<their year> --tag-and-move`	It asks *Move N document pictures?*; answer **y**: they're in `documents\organized\<year>\<year-month>` with `doc_` names, with sidecars	Time: 00:00:00
K22	`da intake undo "<the ocr-moved.csv it printed>"`	They're back in `photos\organized`, with their old names	Time: 00:00:00
K23	`da rename`	*0 would be renamed, N already named so*; *Nothing changed*; the path of `rename-plan.csv`	Time: 00:00:00
K24	`da rename photos\organized\<their year> --pattern "{YYYY}{MM}{DD}_{camera}_{owner}_{hash:4}"`	How many *would be renamed*, raw files following them, and *had nothing for {owner}* for photos with no known owner; the files keep their names	Time: 00:00:00
K25	Open that `rename-plan.csv` in Excel	One row per file: its path, the new name, *rename* / *follows* / *unchanged*, and notes like *the name token {owner} was missing*	Time: 00:00:00
K26	`da rename intake`	*use da run*; nothing is planned	Time: 00:00:00
K27	In the menu press **N**	The photo, video and document patterns, and how to try one	Time: 00:00:00
K28	Type **1**, then the pattern `{YYYY}-{MM}-{DD}_{hh}{mm}_{hash:4}`	How many *would be renamed*; *Rename N files?* - answer **y**	Time: 00:00:00
K29		*Renamed N*, and an *Undo: da intake undo ...* line; then *Save it in da-archive.yaml?* - answer **y**	Time: 00:00:00

#	Do this	You should see	Result
K30	Open a month folder and `da-archive.yaml`	Photos named like `2024-07-04_1015_ab12.jpg` (raw files too, sidecars beside them); in the file, the new pattern and `# was (<today>): photo_...` under it	Time: 00:00:00
K31	Press **N** again, then **Enter**	*0 would be renamed*	Time: 00:00:00
K32	`da intake undo "<the rename-moved.csv it printed>"`	The old names are back; **N**, **Enter** now offers to rename them again (the saved pattern stays until you change it)	Time: 00:00:00
K33	Press **N**, **1**, type the old pattern (`photo_{date_taken:%Y%m%d_%H%M%S}_{hash:4}`), answer **y** twice	Back as before; `da-archive.yaml` keeps both earlier patterns as notes, newest first	Time: 00:00:00
K34	`da rename --folder "{owner}\{YYYY}"`	*N moved to another folder, had nothing for {owner}* for photos with no known owner; nothing moves	Time: 00:00:00
K35	`da rename --folder "{owner}\{YYYY}" --kind photo --apply --save` and answer **y** twice	Photos of cameras with an owner are in `photos\organized\<owner>\<year>` , the others in `photos\organized\<year>` ; the emptied month folders are gone; each folder has a contact sheet	Time: 00:00:00
K36	Copy a new photo into intake and choose **1 Process intake**	It's filed by the new folder pattern	Time: 00:00:00
K37	`da intake undo "<the rename-moved.csv it printed>"` , then **N**, **1**, Enter, type `{YYYY}\{YYYY}-{MM}` , answer **y** twice	Back in year and month folders; `da-archive.yaml` keeps `# folder was (...)` notes	Time: 00:00:00
K38	*(Optional: do it between K35 and K37, while the photos are in the owner layout)* `da location group <a month with GPS photos, e.g. 2024-07>` and give group 1 a title	The month's photos are found in `<owner>\<year>` ; the location folder is made there (`<owner>\<year>\<year-month> <title> (<place>)`)	Time: 00:00:00
K39	Make a text file `logins.txt` in `intake\docs` with the line `Password: Test-1234` , and a PDF that only says *change your password*	(setup)	Time: 00:00:00
K40	`da find --passwords`	*1 hold a password (or PIN), 1 only mention one*; the snippet shows dots, never `Test-1234` ; *Nothing moved*	Time: 00:00:00
K41	Open `passwords.csv` from `output\runs\<date-time>-passwords\`	Both files, the password hidden	Time: 00:00:00

#	Do this	You should see	Result
K42	`da find --passwords --move`, answer **y**	*Moved 1 to ..._secure-holding*; `logins.txt` is in `_secure-holding\<today>\intake\docs`, the PDF stays	Time: 00:00:00
K43	`da intake undo "<the secure-moved.csv it printed>"`	`logins.txt` is back in `intake\docs`	Time: 00:00:00
K44	Put photos in `intake\photos\Jones Family Photos\Scans\` and `...\Jones Family Photos\DCIM\100CANON\`, **1 Process intake**, open a sidecar	An `origin` block: `folders [Jones Family Photos, Scans]`	Time: 00:00:00
K45	Its `auto_tags`	`folder:jones-family-photos` and `folder:scans`; none for `DCIM` or `100CANON`	Time: 00:00:00
K46	`da findit tags photos\organized -r folder:jones-family-photos -v`	Both photos	Time: 00:00:00
K47	Add `origin:` / `skip: [Scans]` to `da-archive.yaml`, `da tag photos\organized --yes`	`folder:scans` gone; `folder:jones-family-photos` kept	Time: 00:00:00
K48	`da rename --folder "{YYYY}\{YYYY}-{MM}-{from:1}"`	The plan puts them in `2009\2009-01-Jones Family Photos ...`; nothing moves	Time: 00:00:00
K49	In an archive filed before this version: `da tag`, then open a sidecar	An `origin` block from the path it came in with (`recorded_at` today)	Time: 00:00:00
K50	**5 Refresh index**, `da db query "SELECT origin, count(*) FROM files GROUP BY origin"`	The folders photos came in from, with counts	Time: 00:00:00
K51	`da ocr` in an archive with filed documents (a PDF, a scan, a letter with a date)	*Read N ... N have dates in them ... N hold a secret*; `ocr-facts.csv` in `output\runs\<date-time>-ocr-facts\`	Time: 00:00:00
K52	Open a document's `.meta.json`	A `dates` block (`found`: the dates, oldest first) and a `secrets` block (`found` true/false) - no text, no secret	Time: 00:00:00
K53	`da ocr` again	*Read 0, skipped N*	Time: 00:00:00
K54	Change one document (or `--again`)	Only that one is read again	Time: 00:00:00

L. Places and location folders (menu L) · Time: 00:00:00 0 / 33

These need an internet connection, except L12.

#	Do this	You should see	Result
L1	Terminal: `da location lookup`	How many photos have a GPS position, how many new places, "about one a second", a privacy note; "Start?"	Time: 00:00:00
L2	Answer **y**	A progress count, then "N of N GPS photos have a place" and a list like `place:springfield-il 24`	Time: 00:00:00
L3	Run it again	"0 new spots"; it finishes at once (places are remembered)	Time: 00:00:00
L4	`da findit tags photos\organized -r place:<one of the places> -v`	Those photos	Time: 00:00:00
L5	`da db query "SELECT city, COUNT(*) FROM files GROUP BY city"`	Towns with counts	Time: 00:00:00
L6	In the menu press **L**	"Months with GPS photos (newest first):" numbered	Time: 00:00:00
L7	Type a month's number	Groups of photos taken close together: count, days and times, places, first file	Time: 00:00:00
L8	For group 1 type a title like `Beach day`; press Enter for the others	"DA will: make 2021-06 Beach day () (N photos)" and "Move them?"	Time: 00:00:00
L9	Answer **y**	"Moved N photos into 1 folder; … contact sheets made" and an **Undo** line	Time: 00:00:00
L10	Open the year folder	`2021-06 Beach day (<place>)` beside `2021-06`, with its own contact sheet; photos without GPS stayed in `2021-06`	Time: 00:00:00
L11	`da validate "photos\organized\2021\2021-06 Beach day (<place>)"`	"Problems found: 0" (a location folder is a proper home)	Time: 00:00:00
L12	`da location rename "<that folder>" "Beach with grandma"`	The new name, "Rename it?", then "Renamed; title set on N sidecars" and an Undo line	Time: 00:00:00
L13	`da location set "<a photo without GPS>" --place "Springfield, IL"`	"Found: Springfield, …", asks, "Done: 1 photo"	Time: 00:00:00
L14	`da location set "<another photo>" --same-as "<a GPS photo>"`	"Done: 1 photo" (no lookup)	Time: 00:00:00
L15	`da location set "<that photo>" --clear`	"Done: 1 photo"	Time: 00:00:00

#	Do this	You should see	Result
L16	Group that month again with **L**	A group taken at the same time and place says "fits the folder already there"; Enter adds to it	Time: 00:00:00
L17	`da location group photos\organized\<year>\<month> --hours 1 --km 0.5`	Smaller groups; answer **n** / press Enter to move nothing	Time: 00:00:00
L18	`da intake undo "<the file L9 named>"`	The photos go back to `2021-06` ; the location folder is gone	Time: 00:00:00
L19	`da location lookup --offline`	Uses only remembered places; nothing sent	Time: 00:00:00
L20	In `da-archive.yaml` set `location:` `lookup: off` , run `da location lookup`	"Lookups are off in da-archive.yaml ..."; set it back afterwards	Time: 00:00:00
L21	`da location group C:\DA-test` (not a month folder)	A clear message that it isn't a month folder	Time: 00:00:00
L22	Add `untitled: '{city}'` under `location:` in `da-archive.yaml` ; **L** on a month with GPS photos	Each group's answer shows its city (e.g. *Springfield*); Enter makes `<YYYY-MM> Springfield (springfield-il)`	Time: 00:00:00
L23	Same month again, type `-` for a group	That group stays in the month folder	Time: 00:00:00
L24	`untitled: '{camera}-{owner}'` , **L** on a month (cameras with owners: `da tag --cameras`)	Folders like `<YYYY-MM> Canon PowerShot A80-jane (...)`	Time: 00:00:00
L25	`da location group <month>` (no `--yes`), then with `--yes`	First *using location.untitled ... Nothing moved*; then the folders are made	Time: 00:00:00
L25a	`untitled: '{camera}-{owner}'` , a month with groups in different towns from the same camera, Enter for each	One folder per place (`<YYYY-MM> <camera> (town-a)` , `(town-b)` ...), not one folder for all	Time: 00:00:00
L25b	`da location group <month> --set-title "Trip"` (a month with groups in two places)	No question per group; *Every group: titled '*Trip*'*; folders `<YYYY-MM> Trip (place-a)` and `(place-b)`	Time: 00:00:00
L25c	`da location group <month> --skip-title` (with `untitled` set)	No question per group; the untitled names, a folder per place	Time: 00:00:00
L25d	`--skip-title` with no `untitled` in `da-archive.yaml`	*...isn't set in da-archive.yaml ... Nothing moved*	Time: 00:00:00
L25e	`da location group <year> --set-title "Trip" --yes` (a year with GPS photos in two months)	*2 months with GPS photos to group: ...*, then each month's folders; another year's photos stay	Time: 00:00:00

#	Do this	You should see	Result
L25f	The same again	*No GPS photos in left to group*	Time: 00:00:00
L25g	**L**, type a year	Each of its months in turn	Time: 00:00:00
L26	Take `untitled` out again	Enter leaves groups in the month folder, as before	Time: 00:00:00

M. Split by years (menu S) · Time: 00:00:00 0 / 22

Copy the kit's `old` photos (another decade) into jane-archive-202x's `intake\photos\old`, process intake (they stay: outside this archive's years). sam-archive-200x (from B12) takes 2000-2009.

#	Do this	You should see	Result
M1	In jane-archive-202x press **S**	"Working out the dates …", "N files in intake were taken outside 2020-2029", and which archive each belongs in	Time: 00:00:00
M2		"Move them to:" with sam-archive-200x listed; choose it	Time: 00:00:00
M3		What will move (from → to) and "Move them?"; answer **y**	Time: 00:00:00
M4		"Moved N files", an Undo line, and "Next: open sam-archive-200x … 1 Process intake"	Time: 00:00:00
M5	Open sam-archive-200x's `intake\photos\from-jane-archive-202x`	The old photos (and their sidecars)	Time: 00:00:00
M6	In sam-archive-200x, process intake	They're filed in `2000s` year folders	Time: 00:00:00
M7	Terminal (jane-archive-202x): `da split`	"0 files in intake were taken outside 2020-2029"	Time: 00:00:00
M8	Undo instead (repeat M1-M4 with new old photos, then `da intake undo "<the file M4 named>"`)	They're back in jane's intake (works even across drives)	Time: 00:00:00
M9	A decade with no archive yet	"N from the 1990s have no archive on this PC yet. Make one: da setup --years 1990-1999"	Time: 00:00:00
M10	Make an archive with no years (`da setup D:\test-all --yes`), copy photos from two decades into its intake, process it	They're filed in its `photos\organized`	Time: 00:00:00
M11	In it: `da split --organized`	*N filed files were taken with a sure date*, and how many belong in each decade's archive	Time: 00:00:00
M12	`da split --organized --to <one decade's archive>` , answer **y**	*N straight into its organized folders*; *Moved N files*; the photos are in that archive's `photos\organized\<year>\<year-month>` with sidecars and a contact sheet, and gone (with their emptied folders) from the first	Time: 00:00:00
M13	Open that archive and choose **2 Status**	The photos are counted there (its index was updated)	Time: 00:00:00
M14	`da intake undo "<the split-moved.csv it printed>"` , then **5 Refresh index** in both	They're back in the first archive	Time: 00:00:00

#	Do this	You should see	Result
M15	Make an archive and merge another into it (`da merge --from <the other>` in the new one); then, in a terminal outside both: `da archives delete <the other>`	*N have a copy elsewhere*, *Everything ... also exists elsewhere*, *Type the archive's name*; press **Enter**: *Kept*	Time: 00:00:00
M16	Copy one new photo into the other archive's `intake\photos` , then `da archives delete <the other>` again	*Not deleted: 1 files exist only in ...*, with that photo listed	Time: 00:00:00
M17	Delete the new photo; `da archives delete <the other>` and type its name	It's in the Recycle Bin, and gone from `da archives`	Time: 00:00:00
M18	Restore it from the Recycle Bin; open its `Open DA.cmd`	It's back on `da archives`	Time: 00:00:00
M19	Close DA; in a terminal outside it: `da archives move D:\<test archive> D:\moved\<test archive>` (same drive), answer **y**	*one rename on the same drive*; the folder is at the new place; `da archives` lists the new place	Time: 00:00:00
M20	Open it there (`Open DA.cmd`), choose **2 Status** and **4 Undo last set-aside**	The counts are as before; the undo finds its files at the new place	Time: 00:00:00
M21	`da archives move <it> E:\<test archive>` (another drive or a USB stick), answer **y**, then **n** to the Recycle Bin question	*The copy is right: N files, M re-read in full*; the old folder has `MOVED.txt` and no `da-archive.yaml`	Time: 00:00:00
M22	`da archives move <it> <a folder that has files in it>`	*isn't empty*; nothing moves	Time: 00:00:00

N. Merge two copies (menu M) · Time: 00:00:00 0 / 8

Copy the whole `C:\DA-test\jane-archive-202x` folder to `C:\DA-test\jane-copy` . Add 3 new photos to jane-copy's intake and process them **in jane-copy** (open its `Open DA.cmd`).

#	Do this	You should see	Result
N1	In **jane-archive-202x** press **M**	This PC's other archives listed; choose jane-copy (or type its folder)	Time: 00:00:00
N2		"Indexing …", "Comparing …", "N files in jane-copy: M already here" and what would be copied (3 to the same folder and name here)	Time: 00:00:00
N3		"Copy them?"; answer **y**	Time: 00:00:00
N4		"Copied 3 files: 3 filed …" and an Undo line; the 3 photos are in jane-archive-202x's month folders	Time: 00:00:00
N5	Look in jane-copy	Unchanged (merge only copies from it)	Time: 00:00:00
N6	Press **M** again, choose jane-copy	"Nothing to copy"	Time: 00:00:00
N7	`da intake undo "<the file N4 named>"`	The 3 copies are removed from jane-archive-202x	Time: 00:00:00
N8	`da merge --from C:\DA-test\nowhere`	A clear message that it isn't an archive	Time: 00:00:00

O. Report and dashboard (menu R, B) · Time: 00:00:00 0 / 22

#	Do this	You should see	Result
O1	Press **R**	A short summary, then the PDF opens	Time: 00:00:00
O2		One page: what's filed (photos, documents, videos, space, years), waiting for you, by year, cameras, places	Time: 00:00:00
O3	Open the archive's `output` folder	`jane-archive-202x-report-YYYY-MM-DD.pdf`	Time: 00:00:00
O4	`da report --refresh`	The index is refreshed first; the same day's PDF is replaced	Time: 00:00:00
O5	`da report --open`	The PDF opens	Time: 00:00:00
O6	Press **B**	The browser opens a page titled *DA - jane-archive-202x*; DA says how to stop it (`da dashboard --stop`) and the menu is back straight away	Time: 00:00:00
O7		**Filed**, **Waiting in intake** (folder by folder), **Needs review**, **Latest runs** and **Activity** match what you've done so far	Time: 00:00:00
O8	Wait a few seconds	**In the archive** fills in: by year, cameras, places (the same numbers as the report)	Time: 00:00:00
O9	Copy a few photos into `intake\photos`, then in another window `da db build`	Within about 10 seconds the page shows the new files waiting; while `da db build` runs, **Happening now** names it	Time: 00:00:00
O10	Press **Recount**	The button greys out, then the numbers are counted again	Time: 00:00:00
O11	Press **B** again	*already running*, with the same address; the same page opens (not a second dashboard)	Time: 00:00:00
O12	Quit the menu and close its window; reload the page	The dashboard still works	Time: 00:00:00
O12a	In a terminal in the archive, `da dashboard --stop`	*Dashboard stopped*; after a few seconds the page says the dashboard stopped	Time: 00:00:00
O12b	`da dashboard --stop` again	*No dashboard is running for this archive*	Time: 00:00:00

#	Do this	You should see	Result
O12c	`da dashboard --foreground --port 9000`	It prints `http://127.0.0.1:9000/` (or the next free port) and the browser opens it; **Ctrl+C** stops it	Time: 00:00:00
O13	From another computer or a phone, try this PC's name or address with that port	It doesn't open: the dashboard is for this PC only	Time: 00:00:00
O14	In the dashboard, scroll to **Pictures**	Small pictures of the latest filed files; videos as *video* tiles	Time: 00:00:00
O15	**Show**: pick a month	That month's pictures	Time: 00:00:00
O16	Click a photo (an iPhone HEIC one if you have it)	It opens large; left/right arrows go through the month; **Esc** closes it	Time: 00:00:00
O17	Click a video, then a PDF	The video plays; the PDF opens in the browser's viewer	Time: 00:00:00
O18	On a photo: **Set as desktop**	*It's the desktop background now*, and it is	Time: 00:00:00
O19	**Open its folder**	File Explorer opens with the file selected	Time: 00:00:00

P. Update sidecars (menu J) · Time: 00:00:00 0 / 5

#	Do this	You should see	Result
P1	Press **J**	"N sidecars: N up to date (schema 2), 0 older." (everything DA made is current)	Time: 00:00:00
P2	*(If you have an older DA archive)* run J there	How many are older and what would change, then "Upgrade N sidecars?"	Time: 00:00:00
P3	Answer **y**	"Upgraded N sidecars", the backup folder, and a `--restore` line	Time: 00:00:00
P4	Run the `--restore` line	"Put back N sidecars"	Time: 00:00:00
P5	`da sidecar upgrade photos\organized\<year>`	Only that year is looked at	Time: 00:00:00

Q. Checking what's filed (terminal) · Time: 00:00:00 0 / 9

#	Do this	You should see	Result
Q1	`da validate photos\organized`	A summary: files checked, problems found (0 or a few notes), nothing moved	Time: 00:00:00
Q2	Rename a filed photo in Explorer to `test.jpg` (keep its `.meta.json` renamed to match: `test.jpg.meta.json`), then `da validate <its folder>`	A "wrong name" problem for it, nothing moved	Time: 00:00:00
Q3	`da validate <its folder> --mode repair`	It's renamed back to `photo_…`	Time: 00:00:00
Q4	`da validate intake --mode repair`	Refused: repair isn't for intake (use 1 Process intake / `da run`)	Time: 00:00:00
Q5	`da validate C:\DA-test\kit-copy --archive C:\DA-test\jane-archive-202x` (any folder outside)	It checks that folder using jane's settings	Time: 00:00:00
Q6	`da run intake\photos\kit\camera --yes`	That folder alone is processed, no questions	Time: 00:00:00
Q7	`da check "<the preview report path from a run>"`	A check of that report	Time: 00:00:00
Q8	`da verify "<a repair report path>"`	Quick: every filed file in place with its size and date, a sample re-hashed; *ok*	Time: 00:00:00
Q8a	`da verify "<a repair report path>" --full`	Every filed file re-read; its hash matches its sidecar (slower)	Time: 00:00:00

R. Settings (da-archive.yaml) · Time: 00:00:00 0 / 5

Make a copy of `da-archive.yaml` first; put it back at the end.

#	Do this	You should see	Result
R1	Change `years:` to `"2021-2021"`, process a folder with 2022 photos	They wait as "outside this archive's years"	Time: 00:00:00
R2	Add an intake folder: under `intake:` add `phone: photo`, then `da setup <this archive>`	`intake\phone` is created	Time: 00:00:00
R3	Change `photoit:` `pattern:` to `{date_taken:%Y-%m-%d}_{hash:4}` and validate a month with `--mode repair`	Photos renamed like `2021-06-01_ab12.jpg` (change it back and repair again)	Time: 00:00:00
R4	Break the file (delete a `:`), then `da status`	A clear message about da-archive.yaml (no crash)	Time: 00:00:00
R5	Put the copy back, then `da status`	The Status as before	Time: 00:00:00

S. Logs, records and safety · Time: 00:00:00 0 / 24

#	Do this	You should see	Result
S1	Open `logs\da.jsonl` in Notepad	One line per command: "da … started", "… finished in Ns", and lines for what was moved	Time: 00:00:00
S1a	Look at any line	It ends with `"host": "<this computer's name>"`	Time: 00:00:00
S1b	Run `da validate photos\organized`, `da db build` and `da contact-sheets photos\organized`, then look at the end of `da.jsonl`	A `validate`, an `index` and a `contact_sheets` line, each with its counts	Time: 00:00:00
S1c	`da history`	*Reading DA's records*, then *What DA did in* (command summaries, with the computer's name) and *Files changed* (e.g. *filed (da run)*, from intake to organized)	Time: 00:00:00
S1d	Open the `history.csv` it saved (`output\runs\<date-time>-history\`)	The same lines, paths inside the archive	Time: 00:00:00
S1e	`da history` again	Quick (*only what's new*); the same changes	Time: 00:00:00
S1f	Undo something (**4 Undo last set-aside**), then `da history`	Those files are marked *(undone …)*	Time: 00:00:00
S1g	`da history --kind video --since <today>`	Only today's videos (or *None recorded yet*)	Time: 00:00:00
S1h	`da history IMG_xxxx.JPG` (a photo's name before DA renamed it)	Its story: *Now* (its organized path, taken, camera, fingerprint), *Came from*, *History* (*filed (da run)*)	Time: 00:00:00
S1i	`da history --hash <first 8 characters of that fingerprint>`	The same story	Time: 00:00:00
S1j	`da history --hash abc`	Refused: *the first 8 or more characters*	Time: 00:00:00
S1k	Rename that photo (**N Naming**, apply), then `da history <its old name>`	*History* shows *filed*, then *renamed*; *Now* is the new name	Time: 00:00:00
S1l	`da history <a photo with a copy in _duplicates-holding>`	*Copies* lists the held one	Time: 00:00:00
S1m	Same, with `--everywhere` (the same photo in another archive)	*Copies* also lists the other archive's	Time: 00:00:00

#	Do this	You should see	Result
S1n	`da history --duplicates`	Each held copy, when it was set aside, and *a copy of* its filed original	Time: 00:00:00
S1o	`da history nothing-like-this.jpg`	*Nothing found* ... (and the `--everywhere` hint)	Time: 00:00:00
S1p	**5 Refresh index**, then `da history --from 100CANON` (a camera folder you processed; any part of a folder name)	*N files in* , counts by camera and by the folder they came from, then the files with where each is now	Time: 00:00:00
S1q	`da history --camera <part of a camera's name, e.g. iphone>`	Only that camera's files (`da tag --cameras` lists the names)	Time: 00:00:00
S1r	`da history --camera <camera> --name <part of one photo's old name>`	Exactly one: its whole story	Time: 00:00:00
S1s	`da history --from NOTAFOLDER`	*Nothing found for from* '*NOTAFOLDER*'	Time: 00:00:00
S2	Open `output\runs\`	A folder per run (`…-run` , `…-split` , `…-merge` , `…-location-group` , ...) with its records	Time: 00:00:00
S3	Start processing a big folder and press **Ctrl+C** during step 2	It stops; run it again and it carries on (nothing half-moved)	Time: 00:00:00
S4	Run `da run --yes` in one window and `da status` in another	Status works while the run is going (don't run two changing commands at once - see the known problems)	Time: 00:00:00
S5	Throughout: did DA ever delete a photo?	No (it only moves, and asks first)	Time: 00:00:00

T. Another PC or a USB drive · Time: 00:00:00 0 / 4

#	Do this	You should see	Result
T1	Copy `C:\DA-test\sam-archive-200x` to a USB stick		Time: 00:00:00
T2	On another PC with DA installed (or the same one, from the stick), double-click the stick's `Open DA.cmd`	The menu opens for that archive	Time: 00:00:00
T3	`da archives`	The stick's archive is now on this PC's list	Time: 00:00:00
T4	Unplug the stick, `da archives`	It's left out until it's plugged in again	Time: 00:00:00

U. Messages when something's wrong · Time: 00:00:00 0 / 6

#	Do this	You should see	Result
U1	`da run` in `C:\` (not an archive)	"Not inside an archive ..." and how to open one	Time: 00:00:00
U2	`da tag photos\organized\2099\2099-01` (a folder that doesn't exist)	*(Known problem: it's treated as a tag. See the known problems; undo with `da tag --remove photos-organized-2099-2099-01`.)*	Time: 00:00:00
U3	`da location set <a photo> --place "zzzz nowhere"`	"Nothing found for ..."	Time: 00:00:00
U4	Disconnect the internet, `da location lookup` on new GPS photos	It stops after two tries and says N photos still need a place; run again later	Time: 00:00:00
U5	`da contact-sheets C:\nowhere`	"No such folder"	Time: 00:00:00
U6	`da --help` and `da location --help` outside an archive	Help shows (no "not inside an archive")	Time: 00:00:00

X. Picasa (da picasa) · Time: 00:00:00 0 / 37

Needs a folder Picasa looked at, with its `Picasa.ini` (or hidden `.picasa.ini`) and photos with named faces.

ID	Do this	Expect	✓
X1	Copy the folder into `intake\photos\`, **1 Process intake**	The photos are filed; the `Picasa.ini` stays in intake	Time: 00:00:00
X2	`da picasa`	*N Picasa.ini files*, *N photos found: N faces (N named, N people)*; nothing written	Time: 00:00:00
X3	Open the `picasa-plan.csv` it names	A row per photo: the filed photo, the people, *followed through DA's moves*	Time: 00:00:00
X4	`da picasa --apply`	*Picasa details written for N photos*; the `Picasa.ini` is now in `picasa\photos\<folder>\`	Time: 00:00:00
X5	Open a filed photo's `.meta.json`	A `picasa` block: faces with `box` and `name` (no emails); `person:<name>` in `auto_tags`	Time: 00:00:00
X6	`da findit tags photos\organized -r person:<name> -v`	That person's photos	Time: 00:00:00
X7	`da tag <that month folder> --yes`, then open the sidecar again	`person:` tags still there	Time: 00:00:00
X8	`da picasa` again	The same photos found (it reads the kept `picasa\` copy)	Time: 00:00:00
X9	`da intake undo "<…\picasa-moved.csv>"`	The `Picasa.ini` is back in intake	Time: 00:00:00
X10	A photo that was a duplicate (set aside) with a face in Picasa	Its details go to the copy that was kept (*the copy kept*)	Time: 00:00:00
X11	`da contact-sheets <a month with Picasa faces>`, open `_contact-sheet.pdf`	A yellow box on each face, the name above it; page 1 has a **People** line	Time: 00:00:00
X12	The same with a photo taken sideways (portrait)	The box is still on the face	Time: 00:00:00
X14	Put a photo and a smaller copy of it (e.g. Picasa's 800 x 600 export) in intake, **1 Process intake**	Step 3: *1 resized copies (e.g. 800 x 600 of a 3072 x 2304 picture)*, a sample check, a question; the small one goes to `_duplicates-holding\<date>\resized\`, the big one is filed	Time: 00:00:00
X15	`da similar` in an archive filed before this	*N resized copies* and a list; nothing moves	Time: 00:00:00
X16	`da similar --apply`	A sample check, a question, *Moved N* and an Undo line	Time: 00:00:00
X17	`da history --duplicates`	The resized ones too, each *a copy of* the filed original	Time: 00:00:00

ID	Do this	Expect	✓
X18	`da share <a month with copies in holding>`	*N copies in holding*; each picture in `details.txt` lists *Copy: exact/resized copy in holding*; the PDF ends with **Copies in holding** pages	Time: 00:00:00
X19	The same with `--no-copies`	No copies listed or shown	Time: 00:00:00
X20	`da intake undo "<…\similar-moved.csv>"`	The resized copies are back where they were	Time: 00:00:00
X21	A photo and the same picture saved again without its date (e.g. by Photoshop Elements) in intake, process	Step 3: *1 re-saved copies*; the undated one goes to `_duplicates-holding\<date>\resaved\`, the dated one is filed	Time: 00:00:00
X22	`da similar` in an archive with Picasa crops	*N pictures with edited or cropped versions* and `versions.csv`	Time: 00:00:00
X23	`da similar --apply`	*Versions linked: N sidecars*; each version's `.meta.json` has a `versions` block (role original / cropped)	Time: 00:00:00
X24	The month's `_contact-sheet.pdf`	Tiles say *+1 version (cropped)* / *cropped version - original filed too*, and *N copies in holding (1 resized …)*	Time: 00:00:00
X25	`da share` that month	The last pages: a framed box per picture with its copies and versions, each labelled	Time: 00:00:00
X26	A burst (two shots a second apart of the same scene)	Neither set aside nor linked	Time: 00:00:00
X27	A copy that had Photoshop Elements keywords, set aside	The kept photo's sidecar has a `keywords` block and those words in `user.tags`	Time: 00:00:00
X28	`da similar`	*N bursts (M photos …) and K blurry photos - marked, never moved*; `bursts.csv`, `blurry.csv`	Time: 00:00:00
X29	Open `bursts.csv` and look at a burst's photos	Shots of the same thing, a few seconds apart; one marked *best*	Time: 00:00:00
X30	Open `blurry.csv` and look at its photos	Out of focus or accidental shots (not just foggy or plain ones)	Time: 00:00:00
X31	`da similar --apply`	Nothing moves; … *marked: N sidecars (tags burst, burst:best, blurry)*	Time: 00:00:00
X32	`da findit tags photos\organized -r burst:best -v`	The sharpest shot of each burst	Time: 00:00:00

ID	Do this	Expect	✓
X33	A month's `_contact-sheet.pdf` with a burst	Tiles say *burst 1 of 3*, *burst 2 of 3, best ...*; a blurry one *blurry*	Time: 00:00:00
X34	`da tag <that month> --yes`, then a sidecar again	`burst` / `blurry` tags still there	Time: 00:00:00
X35	`da share` that month	`details.txt`: *Burst: shot 2 of 3 taken one after another (the sharpest ...)*, *Blurry: ...*	Time: 00:00:00
X36	Open a filed photo in Windows Photos, make it much brighter (or black and white), **Save as copy** into `intake\photos\edits\` as `<its name>-edited.jpg`	The filed photo is unchanged	Time: 00:00:00
X37	**1 Process intake**	The edit is filed in the same month folder, with a new name (same date and time, a new 4-letter ending)	Time: 00:00:00
X38	`da similar --apply`	*Versions linked*; both sidecars have a `versions` block (role original / edited)	Time: 00:00:00

Y. People (da facial) · Time: 00:00:00 0 / 27

Needs an archive where `da picasa --apply` has brought in Picasa's named faces (section X).

ID	Do this	Expect	✓
Y1	`da facial scan` (the first time)	*Downloading face_detection_yunet ... (once)* and *face_recognition_sface ... (39 MB, once)*, then *N photos; N not looked at yet*	Time: 00:00:00
Y2	Let it run (asks first when there are more than 500)	*Looked at N photos: N faces ... Faces with Picasa's names: N*	Time: 00:00:00
Y3	`da facial scan` again	*0 not looked at yet*; quick	Time: 00:00:00
Y3a	`da facial scan` on a big archive (10,000+ photos)	It keeps going at a steady pace (about 0.1 s a photo); Task Manager shows DA staying under about 1 GB of memory	Time: 00:00:00
Y4	`da facial people`	A table: each person, photos, *Named by Picasa*, *Suggested* (0 until suggest)	Time: 00:00:00
Y5	`da facial suggest`	*Suggested: N faces probably someone known, N maybe; N not like anyone known (N groups)*, a CSV and a PDF	Time: 00:00:00
Y6	Open the PDF	A page per person with faces suggested (*probably 0.62*); *Who is this? Group 1* pages of alike faces	Time: 00:00:00
Y7	`da facial people` again	*Suggested: probably / maybe* counts filled in	Time: 00:00:00
Y8	A person shown as an account id (`..._lh,4d61...`)	Marked *(a Picasa contact with no name)* with the rename line	Time: 00:00:00
Y9	`da facial rename "<old>" "<new>"`	*Renamed N faces; N sidecars changed*; their `person:` tags have the new name	Time: 00:00:00
Y10	`da facial forget --person "<someone>"`, then `da facial people`	They're gone from the list (Picasa's names in sidecars stay)	Time: 00:00:00
Y11	On a PC with Smart App Control on: `da facial scan`	It works, or says plainly that Windows blocked it; other `da` commands still work	Time: 00:00:00
Y12	`da dashboard`, **People** - **Show**	*N photos looked at, N faces, N people*; Everyone, Waiting for you (faces with *probably 0.62*), Who is this? groups	Time: 00:00:00
Y13	Under a person, click a face that isn't them	It turns red with an X	Time: 00:00:00
Y14	**Confirm the ticked ones**	*N faces named ...*, then *N faces: not them*; the person's count of photos goes up, the waiting faces go	Time: 00:00:00

ID	Do this	Expect	✓
Y15	Open one of those photos' `.meta.json`	A `faces` block with the name and `"source": "you"`; `person:<name>` in `auto_tags`	Time: 00:00:00
Y16	A group: type a name (the box offers known names), **Name the ticked ones**	The group goes; that person is in Everyone	Time: 00:00:00
Y17	Another group: **Not a face**	It goes, and doesn't come back after `da facial suggest`	Time: 00:00:00
Y18	Click a name in Everyone, type a new name	*is now*; their photos' tags have the new name	Time: 00:00:00
Y19	`da facial suggest` again, then **Refresh**	The faces you turned down aren't suggested for that person again	Time: 00:00:00
Y20	`da tag <a month with named faces> --yes`	The `person:` tags you gave are still there	Time: 00:00:00
Y21	After confirming faces on the People page	… *Their contact sheets are being updated*; a moment later that month's `_contact-sheet.pdf` shows the names in yellow on those faces, and page 1's **People** row has them	Time: 00:00:00
Y22	A photo Picasa named and you confirmed too	One yellow box on that face, not two	Time: 00:00:00
Y23	`da share` that month	The names on the big pictures and in *People:* on page 1 and the email	Time: 00:00:00
Y24	`da history <one of those photos>`	A *People:* line with the names	Time: 00:00:00
Y25	**1 Process intake**: switch on **12** and process a folder	*12/12 Faces: N photos looked at, N faces …; suggested: …*; the new faces are on the People page	Time: 00:00:00
X13	`da share <that month> --to <your own list>`	Page 1: *People:* … and *Tags:* …; film strips with boxes; big pictures with boxes and names; `details.txt` starts with the people and each picture lists *People* and *Tags*	Time: 00:00:00

W. Share with family and friends (da share) · Time: 00:00:00 0 / 28

Sharing sends email **from your own Gmail**, so it needs a one-time setup in Google's developer pages (about ten minutes; the guide's *Set up sharing (once)* has the same steps). Use a list with only **your own address** while testing, so nobody else gets test emails. Google's screens change now and then: if a name below doesn't match, look for

the nearest one (newer pages call the consent screen **Google Auth Platform**, with **Branding**, **Audience** and **Clients**).

#	Do this	You should see	Result
W1	In `da-archive.yaml` add, at the end: `share:` / `lists:` / `me: [you@gmail.com]` (your own address); then `da share --lists`	*me: 1 people*	Time: 00:00:00
W2	`da share photos\organized\<a month with photos> --to me --note "Test share"`	*Making the sheet: N photos* (videos and documents left out are counted), the PDF's folder in `output\shares\` , and *Nothing sent. To send it to me (1 people): add --send*	Time: 00:00:00
W3	Open that PDF	Page 1: the title, *N pictures*, the dates, *From the archive ()*, your note, then every picture on film strips (4 a strip), each frame numbered along the top and with an ID like ▶ *9B05* along the bottom; then 6 big pictures a page with the date, place and the same ID under each	Time: 00:00:00
W4	The same command with `--send` (before any setup)	*No Google client file at ...\google-client.json. See 'Set up sharing' in the guide*; nothing sent	Time: 00:00:00
W5	Open https://console.cloud.google.com, sign in with your Gmail; at the top, **Select a project** > **New project**, give it a name of 4 to 30 characters (e.g. `DA-sharing`), **Create**, and select it	The page shows project your project at the top	Time: 00:00:00
W6	**APIs & Services** > **Library**, search **Gmail API**, open it, **Enable**	*Gmail API* with **Manage** (it's enabled)	Time: 00:00:00
W7	**APIs & Services** > **OAuth consent screen** (or **Google Auth Platform**): **External**; app name `DA-sharing` (or your project's name); your email where asked; save	The consent screen is set up, publishing status **Testing**	Time: 00:00:00
W8	On the same pages, **Test users** (or **Audience**) > **Add users**: your own Gmail address; save	Your address is listed as a test user	Time: 00:00:00
W9	**Credentials** (or **Clients**) > **Create credentials** > **OAuth client ID**; application type **Desktop app**, any name (e.g. `Desktop client 1`); **Create**; **Download JSON**	A file like `client_secret_...json` in Downloads	Time: 00:00:00
W10	In File Explorer type `%LOCALAPPDATA%\DA` in the address bar; put the downloaded file there and rename it `google-client.json`	`%LOCALAPPDATA%\DA\google-client.json` exists	Time: 00:00:00
W11	In a terminal: `da share --setup`	Your browser opens Google's sign-in: choose your account	Time: 00:00:00

#	Do this	You should see	Result
W12		*Google hasn't verified this app*: choose **Continue** (it's your own app); then allow **Send email on your behalf**	Time: 00:00:00
W13	Back in the terminal	*Signed in to Gmail (DA may only send). The sign-in is kept in ...\google-token.json*	Time: 00:00:00
W14	Look in `%LOCALAPPDATA%\DA`, then search the archive folder for `google`	`google-token.json` is in `%LOCALAPPDATA%\DA`; nothing about Google is in the archive	Time: 00:00:00
W15	`da share photos\organized\<the month> --to me --note "Test share" --send`	*Email 1 message to me (1 people)?*; answer **y**: *Sent Photos: ... to 1 person (each their own email)*	Time: 00:00:00
W16	Open your Gmail inbox	An email *Photos:* , your note, ... *from the archive* , then each picture by its ID: name, file (in your archive), SHA-256, camera, place, GPS, when taken; the PDF attached; the To line has only your address	Time: 00:00:00
W16a	Pick one ID from the email and find it in the PDF	The same ID on its film-strip frame and under its big picture; `details.txt` beside the PDF has the same list	Time: 00:00:00
W17	Gmail's **Sent** folder	The same email, sent from your address	Time: 00:00:00
W18	Add a second address of yours to the list (`me: [you@gmail.com, you2@example.com]`) and send again	*to 2 people*; each address gets its own email, showing only that address	Time: 00:00:00
W19	Share a few months at once (big enough: 200+ photos), with `--send`	*2 PDFs* (or more) and emails *(1 of 2)*, *(2 of 2)*, each with the note page	Time: 00:00:00
W20	`da share photos\organized\<the month> --to neighbours`	*No list called 'neighbours'*; nothing made	Time: 00:00:00
W21	In the dashboard (menu **B**), open a photo large and press **Share**	(one list: *Email this photo to me?* - **OK**) *Sent to me (1 people)*; the photo arrives as a JPEG	Time: 00:00:00
W22	Open a video or a document large	**Share** is greyed out (photos only)	Time: 00:00:00
W23	Look at the end of `logs\da.jsonl`	`share` lines with the list's name and the counts, and no email addresses	Time: 00:00:00
W24	*(Optional)* On the consent screen, **Publish app** (or **Audience** > **Publish**)	Publishing status **In production**; you won't be asked to sign in again every 7 days	Time: 00:00:00

#	Do this	You should see	Result
W25	Delete `%LOCALAPPDATA%\DA\google-token.json` (that's how to sign out), then share with `--send`	The browser opens to sign in again; after allowing it, the email is sent	Time: 00:00:00
W26	*(Another computer)* Copy the same `google-client.json` into that computer's `%LOCALAPPDATA%\DA`, then `da share --setup` there	It signs in on that computer too (its own `google-token.json`); one OAuth client is enough for every computer	Time: 00:00:00
W27	*(Someone else sends from their own Gmail, on their computer)* Add their address under **Test users** (or **Audience**), or publish the app (W24); they run `da share --setup` and sign in with their account	Their shares come from their address	Time: 00:00:00

V. Clean up · Time: 00:00:00 0 / 2

#	Do this	You should see	Result
V1	Delete `C:\DA-test` and `C:\DA-test-kit` when you're done	(Your real photos were never touched)	Time: 00:00:00
V2	To remove DA: delete `C:\Scripts\DA`, and remove it from your PATH (Settings, search "environment variables", Path)	`da` is no longer recognised	Time: 00:00:00

Reporting a problem

1. Check the known problems first.
2. On the website, press the step's **Report** button: a new issue opens at GitHub with the step, your name and your version filled in (a free GitHub account is needed). Or open one yourself at https://github.com/devmukmuk/DA/issues, or send it to the person who asked you to test. The panel at the top shows how many of your reports are still open.
3. Include: the step number (**L9**), what you did, what you expected, what happened (a screenshot helps), your DA version (`da version`), and whether DA is the DA.exe or the Python install.
4. If DA printed an error, the archive's `logs\da.jsonl` has the details: copy the last few lines.
5. Leave out anything private: names, photos, and folder paths you'd rather not share.